

Q8. Application: Gaming Engine Optimization

Asymptotic Analysis of Rendering Algorithms

Name: Arun Balaji S J Reg. No: 192511075

1. Problem Overview

A gaming engine needs to render frames efficiently, and the choice of rendering algorithm directly determines how well the engine scales as screen resolution (and therefore the number of pixels, vertices, or objects, n) increases. Three candidate algorithms are considered, with time complexities $O(n)$, $O(n \log n)$, and $O(n^2)$. This analysis studies their asymptotic behaviour, compares their performance for high-resolution inputs, explains the impact of growth rate, and justifies the most efficient choice.

2. Asymptotic Analysis of Each Algorithm

2.1 Algorithm A — $O(n)$ (Linear Time)

This algorithm processes each input element (e.g., each pixel or vertex) a constant number of times. As n increases, the running time increases proportionally. Doubling the input doubles the work. This is the ideal growth rate for real-time rendering because performance scales predictably and remains manageable even for very large frame buffers.

2.2 Algorithm B — $O(n \log n)$ (Linearithmic Time)

This algorithm typically arises when the rendering pipeline combines a linear pass with a divide-and-conquer step, such as sorting objects for depth ordering or spatial partitioning (e.g., BSP trees, sorting draw calls). The $\log n$ factor grows very slowly, so for large n the running time stays close to linear. This makes it highly efficient and only marginally slower than a pure $O(n)$ approach.

2.3 Algorithm C — $O(n^2)$ (Quadratic Time)

This algorithm likely compares or interacts every element with every other element (e.g., naive pairwise collision detection or brute-force pixel comparisons). As n grows, the work grows with the square of n . This is acceptable for small n but becomes computationally explosive for large, high-resolution frames, since work grows far faster than the input itself.

3. Performance Comparison for High-Resolution Inputs

High-resolution rendering (e.g., 4K, with roughly 8.3 million pixels per frame, compared to about 2 million for 1080p) dramatically increases n . The table below illustrates approximate relative operation counts as n scales up:

Input Size (n)	$O(n)$	$O(n \log n)$	$O(n^2)$
1,000,000	1,000	1000	~9,966
1,000,000,000,000	1,000,000	1,000,000	~19,931,569
$\approx 6.9 \times 10^{13}$	8,300,000	8,300,000	~187,000,000 (approx.)

The figures show that $O(n)$ and $O(n \log n)$ scale in a controlled, near-proportional manner as resolution increases, while $O(n^2)$ grows explosively — at 4K resolution the quadratic algorithm would require on the order of 10^{13} operations per frame, which is computationally infeasible for real-time rendering (typically requiring 60+ frames per second).

4. Summary Comparison Table

Algorithm	Time Complexity	Growth Behaviour	Suitability for High-Res Input
Algorithm A	$O(n)$	Linear — time grows proportionally with input size	Excellent; scales smoothly to very large n
Algorithm B	$O(n \log n)$	Slightly super-linear — grows a little faster than n	Very good; near-linear performance in practice
Algorithm C	$O(n^2)$	Quadratic — time grows with the square of input size	Poor; becomes unusable as resolution increases

5. Growth Rate Impact

- $O(n)$: Work increases in direct proportion to input size. Predictable and scalable — ideal for real-time systems where frame budgets are fixed (e.g., 16.6 ms for 60 FPS).
- $O(n \log n)$: Work increases slightly faster than linear, but the logarithmic term grows so slowly that performance remains close to linear even for very large n . Efficient for tasks needing ordering or partitioning.
- $O(n^2)$: Work increases with the square of input size. Small inputs are fine, but as n grows (as with higher resolutions), execution time increases dramatically, causing frame drops, stutter, and an unplayable experience.

In practical terms, as resolution increases from standard definition to 4K and beyond, an $O(n)$ or $O(n \log n)$ algorithm continues to deliver frames within the required time budget, whereas an

$O(n^2)$ algorithm's execution time increases so steeply that it quickly exceeds the available frame time, making it unsuitable for high-resolution, real-time rendering.

6. Justification of the Most Efficient Algorithm

Algorithm A, with $O(n)$ time complexity, is the most efficient choice for the gaming engine. Its running time scales directly and predictably with input size, ensuring consistent frame rates even as resolution increases to 4K or beyond. While Algorithm B, $O(n \log n)$, is also highly efficient and may be necessary when the task inherently requires sorting or spatial partitioning, it introduces a small additional overhead compared to pure linear processing. Algorithm C, $O(n^2)$, should be avoided for high-resolution rendering, as its growth rate makes it computationally impractical once n becomes large, leading to unacceptable frame times and poor user experience.

Conclusion: For a gaming engine targeting high-resolution, real-time rendering, the $O(n)$ algorithm is the most efficient overall. If additional functionality such as sorting or spatial ordering is required, the $O(n \log n)$ algorithm remains a strong, scalable alternative. The $O(n^2)$ algorithm is the least suitable and should be replaced or optimized wherever possible.